Yes you should understand backprop

Andrej Karpathy Follow 7 min read · Dec 19, 2016

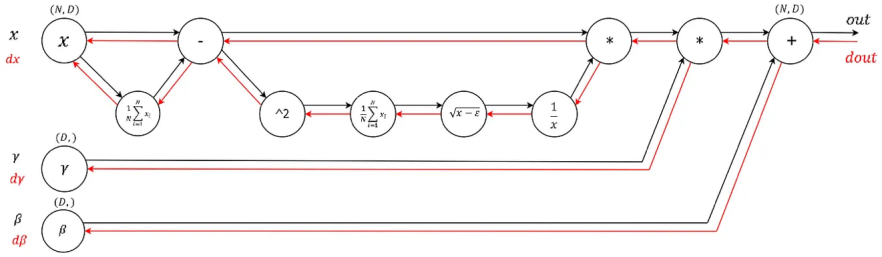
When we offered CS231n (Deep Learning class) at Stanford, we intentionally designed the programming assignments to include explicit calculations involved in backpropagation on the lowest level. The students had to implement the forward and the backward pass of each layer in raw numpy. Inevitably, some students complained on the class message boards:

> *"Why do we have to write the backward pass when frameworks in the real world, such as TensorFlow, compute them for you automatically?"* 1

This is seemingly a perfectly sensible appeal - if you're never going to write backward passes once the class is over, why practice writing them? Are we just torturing the students for our own amusement? Some easy answers could make arguments along the lines of *"it's worth knowing what's under the hood as an intellectual curiosity"*, or perhaps *"you might want to improve on the core algorithm later"*, but there is a much stronger and practical argument, which I wanted to devote a whole post to:

> The problem with Backpropagation is that it is a leaky abstraction.

In other words, it is easy to fall into the trap of abstracting away the learning process — believing that you can simply stack arbitrary layers together and backprop will "magically make them work" on your data. So lets look at a few explicit examples where this is not the case in quite unintuitive ways.



Some eye candy: a computational graph of a Batch Norm layer with a forward pass (black) and backward pass (red). (borrowed from this post)

1

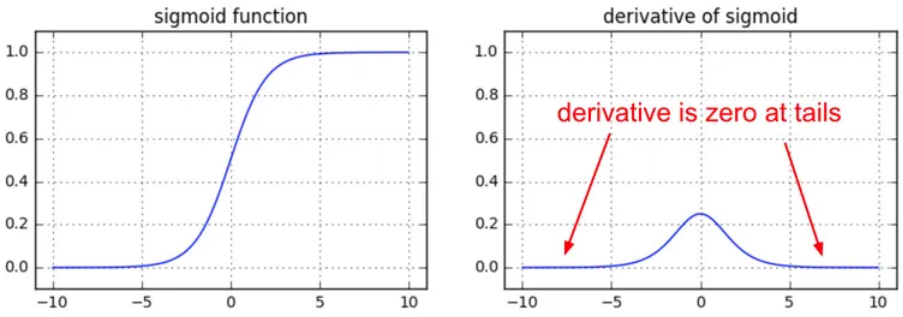
Vanishing gradients on sigmoids

We're starting off easy here. At one point it was fashionable to use **sigmoid** (or **tanh**) non-linearities in the fully connected layers. The tricky part people might not realize until they think about the backward pass is that if you are sloppy with the weight initialization or data preprocessing these non-linearities can "saturate" and entirely stop learning — your training loss will be flat and refuse to go down. For example, a fully connected layer with sigmoid non-linearity computes (using raw numpy):

```
z = 1/(1 + np.exp(-np.dot(W, x))) # forward pass
dx = np.dot(W.T, z*(1-z)) # backward pass: local gradient for x
dW = np.outer(z*(1-z), x) # backward pass: local gradient for W
```

1

If your weight matrix **W** is initialized too large, the output of the matrix multiply could have a very large range (e.g. numbers between -400 and 400), which will make all outputs in the vector **z** almost binary: either 1 or 0. But if that is the case, **z*(1-z)**, which is local gradient of the sigmoid non-linearity, will in both cases become **zero** ("vanish"), making the gradient for both **x** and **W** be zero. The rest of the backward pass will come out all zero from this point on due to multiplication in the chain rule.



Another non-obvious fun fact about sigmoid is that its local gradient (z*(1-z)) achieves a maximum at 0.25, when z = 0.5. That means that every time the gradient signal flows through a sigmoid gate, its magnitude always

diminishes by one quarter (or more). If you're using basic SGD, this would make the lower layers of a network train much slower than the higher ones.

Top highlight

TLDR: if you're using **sigmoids** or **tanh** non-linearities in your network and you understand backpropagation you should always be nervous about making sure that the initialization doesn't cause them to be fully saturated. See a longer explanation in this CS231n lecture video.

Dying ReLUs

Another fun non-linearity is the ReLU, which thresholds neurons at zero from below. The forward and backward pass for a fully connected layer that uses ReLU would at the core include:

```
z = np.maximum(0, np.dot(W, x)) # forward pass
dW = np.outer(z > 0, x) # backward pass: local gradient for W
```

If you stare at this for a while you'll see that if a neuron gets clamped to zero in the forward pass (i.e. **z**=0, it doesn't "fire"), then its weights will get zero gradient. This can lead to what is called the "dead ReLU" problem, where if a ReLU neuron is unfortunately initialized such that it never fires, or if a neuron's weights ever get knocked off with a large update during training into this regime, then this neuron will remain permanently dead. It's like permanent, irrecoverable brain damage. Sometimes you can forward the entire training set through a trained network and find that a large fraction (e.g. 40%) of your neurons were zero the entire time.